Arduino: microcontrolador, pines, GND y el primer programa

Lección 5 — 1MAK, SST-INF-01c — La placa Arduino Uno, los pines digitales, la referencia GND, USB y alimentación, Tinkercad, pinMode, digitalWrite, delay, setup y loop

13/10/2026

1 Desde dónde seguimos

Nota

Volvemos al tema que empezamos las primeras clases hablando de **Arduino**: la **placa electrónica** que nos servirá para aprender a programar. El profesor ha empezado a **fabricar** una placa con los componentes que usaremos en las próximas clases. Antes de pasar a la práctica hacen falta algunas cositas sin las cuales no se puede trabajar: hoy vemos cómo está hecha la placa y escribimos, en simulación, el **primer programa**.

2 El cerebro de la placa: el microcontrolador

Definición

El **microcontrolador** es el **cerebro** de la placa electrónica: el componente que, según el **programa** que escribimos nosotros, controla todo lo demás.

- En el mundo existen **miles de modelos** distintos de microcontroladores (decenas, cientos de miles).
- El que está montado en el **Arduino Uno**, el modelo básico e "histórico", se llama **ATmega328P**. Después salieron muchas otras versiones de Arduino, con procesadores más o menos potentes.
- Es un **chip negro** con muchas **"patitas"** metálicas, los **pines** (o terminales).

Definición

Qué hace un pin. Cada terminal metálico del microcontrolador es un **cable** que podemos **controlar desde el programa**: podemos ponerle una **tensión** o no.

- Arduino trabaja a **5 voltios**. Respecto a una patita de referencia que está a **0 V**, un pin puede estar a **+5 V** o a **0 V**.
- Por lo tanto un pin puede estar **encendido o apagado**: cuando es **0** está a 0 V, cuando es **1** el microcontrolador le pone **5 V**.
- La secuencia de encendidos y apagados depende del **programa** que escribimos.

3 Los conectores del borde y los pines digitales

Las patitas del microcontrolador están **llevadas al borde** de la placa, a unos conectores con agujeritos en los que se pueden meter cables. Ahí conectamos resistores, lámparas, pulsadores, servomotores… todo lo que se os ocurra: es nuestra interfaz hacia el exterior. Qué agujerito está conectado a qué patita del chip, por ahora, nos importa poco.

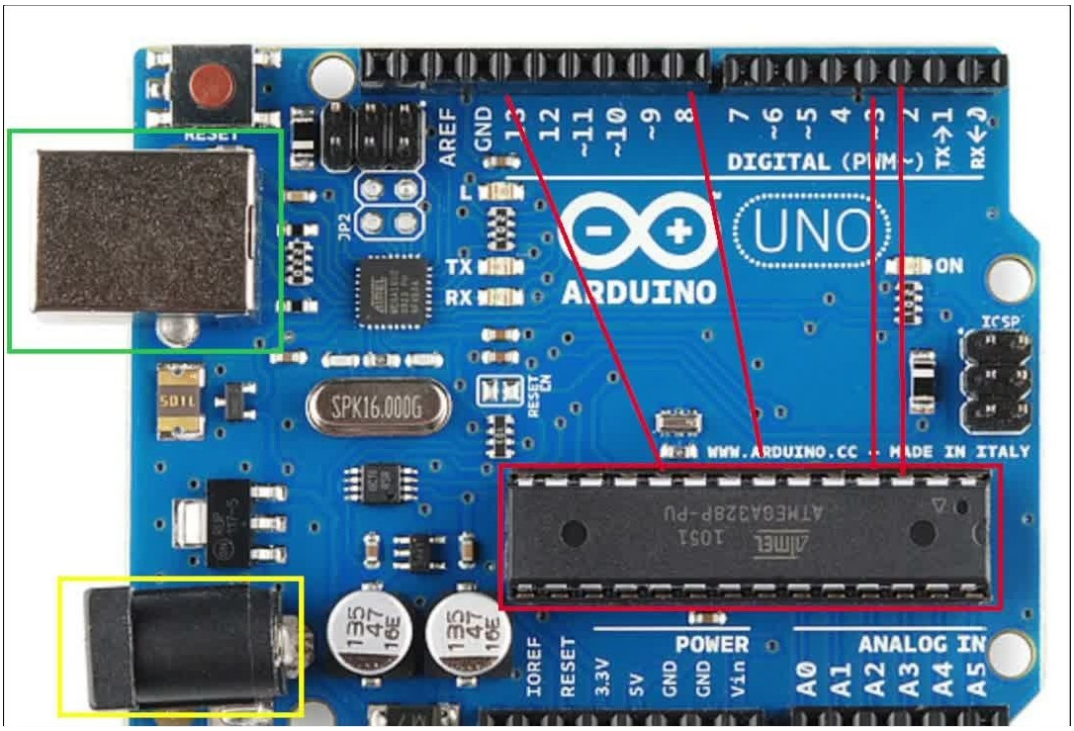


Figura 1: La placa Arduino Uno marcada en clase. En rojo el microcontrolador ATmega328P y algunas de las conexiones entre sus patitas y los conectores de arriba; en verde el puerto USB; en amarillo el conector de alimentación.

Definición

Los pines digitales del 0 al 13. Arriba, empezando por la derecha, los pines están **numerados del 0 al 13**. ¿Cuántos son en total? No 13: son **14**, porque se empieza desde el **cero**.

Nota

En informática el **cero** se escribe a menudo con una **barra en el medio** (Ø), para no confundirlo con la letra O. En la impresión de la placa el cero está un poco estropeado, pero es el cero.

Atención

Los pines 0 y 1 (RX y TX). Al lado de los pines 0 y 1 pone **RX** y **TX**: le sirven a la placa para **comunicarse con el ordenador** y para cargarle el programa, que viaja justamente por esos cables. Se pueden usar también para otras cosas, pero hay que tener cuidado con cosas que todavía no podemos entender: si no las hacemos, algo no funciona y no se entiende por qué. **Regla por ahora:** los pines 0 y 1 **no los usamos**, los dejamos siempre libres. Con el tiempo aprenderemos a aprovecharlos.

4 GND: la referencia de tensión

Definición

La patita **GND** (del inglés *ground*, “tierra”) es nuestra **referencia de tensión**: para la placa es el punto a **0 voltios**.

- En la placa hay **tres**: uno arriba, al lado del pin 13, y dos abajo.
- Si pensamos en un generador, el GND es su **negativo**: el terminal a potencial **0 V**.
- Cuando decimos que un punto “vale 5 V”, queremos decir **5 V respecto a GND**.

Recordad que **electrónica y firmware van juntos**: aun hablando de programación entran cosas de electrónica. En los **esquemas eléctricos** el cable de GND tiene su propio **símbolo**. En un circuito con un generador y un resistor, los símbolos de masa indican que los puntos donde están dibujados están **conectados entre sí** y valen **0 V**, aunque no se dibuje el cable que los une. Si el cable de arriba está a 5 V, todo lo que está conectado ahí está a 5 V **respecto a GND**.

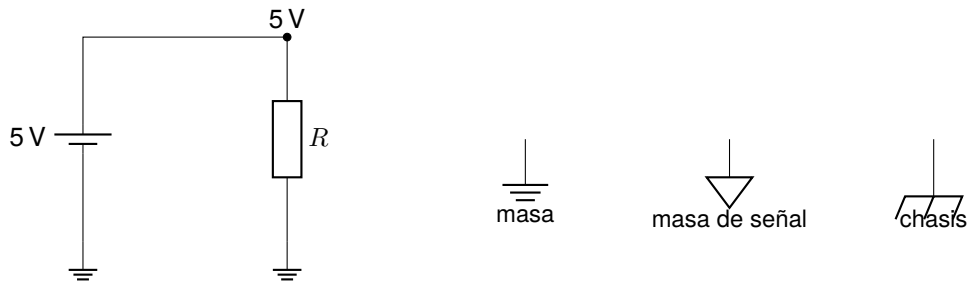


Figura 2: A la izquierda: un generador de 5 V y un resistor; los dos símbolos de masa indican que los dos puntos de abajo están conectados entre sí y están a 0 V. A la derecha: tres símbolos que se encuentran en los esquemas para indicar la referencia a 0 V (reconstrucción: los símbolos dibujados en la pizarra no se ven en la grabación).

Atención

No hay un símbolo único. En los esquemas eléctricos encontráis **símbolos distintos** para el GND, depende de quién dibuja. Algunos símbolos parecidos sirven para decir **otras cosas**: por ejemplo el **símbolo de tierra** (la conexión a la instalación de tierra) no es lo mismo que la referencia a 0 V del circuito. Hay que **aprender a reconocerlos**; a nosotros por ahora nos bastan tres.

Atención

Regla práctica. Cada vez que conectamos un circuito a Arduino, el **GND** tiene que llegar a **todos los periféricos** que usamos: es la referencia de tensión común.

5 USB y alimentación

- El **puerto USB** (en verde en la figura) sirve para **conectar la placa al ordenador**: la placa se comunica con el ordenador y el ordenador le **carga el programa**.

- El **conector de alimentación** (en amarillo) sirve para **alimentar** la placa sin ordenador.

Definición

El ordenador solo sirve para programar. Una vez que el programa está cargado en la **memoria del microcontrolador**, el ordenador **ya no hace falta**. Ejemplo: quiero construir un **dron** (o un sistema para controlar el riego de un viñedo). El dron no puede volar conectado con un cable al ordenador. Entonces:

1. conecto la placa al ordenador con el **cable USB** y cargo el programa;
2. compruebo que funciona;
3. desconecto el cable USB;
4. alimento la placa desde el **conector de alimentación**, por ejemplo con una **batería**: la placa se enciende y, si la hemos programado bien, funciona.

En cambio, cuando experimentamos en el ordenador (por ejemplo para hacer parpadear una lámpara), el cable USB ya lleva también la **alimentación**: la placa la toma de ahí.

6 Dos formas de trabajar: placa real y simulador

- En los próximos días trabajaremos con la **placa real**: Arduino, algunos **resistores**, **lámparas** y **pulsadores**, con los que haremos pequeños proyectos para aprender a usarlo.
- Antes, para no **quemar una placa** conectándola mal, se trabaja en **simulación**, con un simulador muy sencillo que ya hemos usado: **Tinkercad**.

Entrar en Tinkercad

1. Encended el ordenador e iniciad sesión en **Chrome** con vuestra **cuenta** (arriba a la derecha).
2. Id a la página **tinkercad.com** y pulsad **Accedi** (Iniciar sesión).
3. Quien ya se había registrado la otra vez elige el acceso con **cuenta personal** y luego entra con la **cuenta de Google**; quien nunca lo ha hecho primero tiene que **registrarse**.
4. La próxima vez será más rápido. El profesor ve vuestros monitores: si alguien tiene problemas, llama y se resuelve juntos.

El primer circuito

En el aula Tinkercad está en **italiano**: los nombres de los botones se dejan en italiano, con la traducción entre paréntesis.

1. Arriba a la derecha: **Crea** → **Circuito** (Crear → Circuito).
2. De la lista de componentes se toma **Arduino Uno R3**: se mantiene pulsado y se **arrastra** al plano de trabajo. En el simulador dejamos **solo el Arduino**.
3. Arriba se pulsa **Codice** (Código). Lo primero que hay que hacer, **cada vez**, es cambiar de **Blocchi** (Bloques) a **Testo** (Texto) y confirmar con **Continua** (Continuar).

4. En el código aparece tres veces la palabra `LED_BUILTIN`: en los **tres sitios** se borra y en su lugar se escribe el número **13**.
5. Se pulsa **Avvia simulazione** (Iniciar simulación): la lamparita **L** de la placa **parpadea**.

7 El primer programa

Ejemplo

El código en Tinkercad (es código en C; Tinkercad lo indica como C++):

```
// C++ code
//
void setup()
{
  pinMode(13, OUTPUT);
}

void loop()
{
  digitalWrite(13, HIGH);
  delay(1000); // Wait for 1000 millisecond(s)
  digitalWrite(13, LOW);
  delay(1000); // Wait for 1000 millisecond(s)
}
```

Los **comentarios** en inglés los pone Tinkercad: *Wait for 1000 millisecond(s)* = “espera 1000 milisegundos”.

El **13** es el número del **pin** que queremos controlar, es decir el terminal 13. ¿Por qué justamente ese? Porque en la placa Arduino el pin 13 está conectado también a la pequeña lamparita **L**. Al programa le estamos diciendo: **enciende** la lámpara, luego **apágala**, y así sucesivamente.

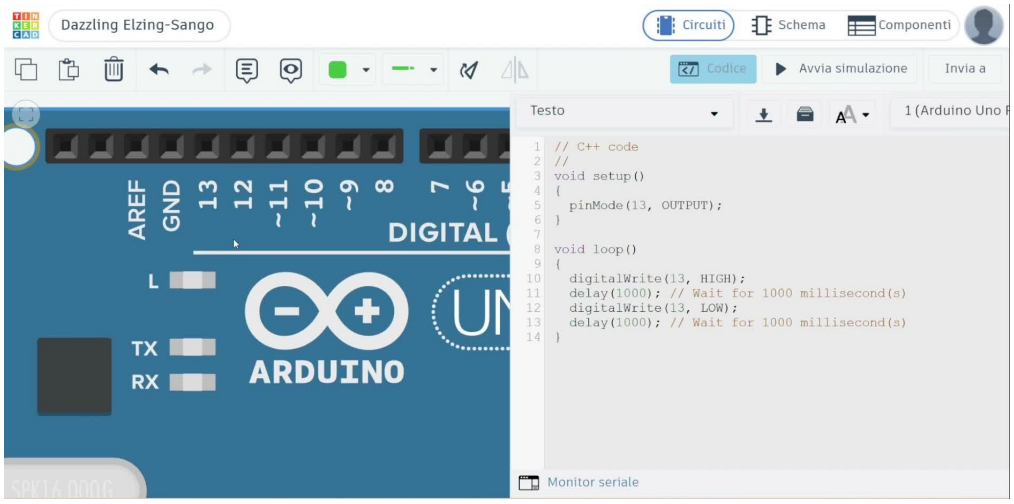


Figura 3: Tinkercad: a la izquierda el Arduino Uno con la lamparita L (cerca del pin 13), a la derecha el código en modo Testo (Texto), con el número 13 en lugar de `LED_BUILTIN`. Arriba los botones Codice (Código) y Avvia simulazione (Iniciar simulación).

7.1 HIGH y LOW

Definición

- `HIGH` quiere decir **alto**: para nosotros **encendido**, a nivel de bit **1** (el pin va a 5 V).
- `LOW` quiere decir **bajo**: para nosotros **apagado**, a nivel de bit **0** (el pin va a 0 V).

HIGH y LOW son los dos **estados** posibles de un cable, es decir de un pin: encendido o apagado.

7.2 digitalWrite: escribir el estado de un pin

Definición

El comando `digitalWrite` sirve para **escribir** el estado de un pin: decide si el pin está **alto o bajo**, encendido o apagado.

```
digitalWrite(13, HIGH);   // pin 13 encendido
digitalWrite(13, LOW);    // pin 13 apagado
```

Entre los paréntesis: primero el **número del pin**, luego una coma, luego `HIGH` o `LOW`. La línea termina con el **punto y coma**.

Ejemplo

Preguntas de la clase.

- *¿Y si escribo 6 en lugar de 13?* El programa funciona igual: el pin 6 se enciende y se apaga. Pero en el simulador **no se ve nada**, porque la lámpara está conectada solo al **13**.
- *¿El signito al lado de algunos números (11, 10, 9…)?* Es la **tilde** (~): indica una función más avanzada que veremos más adelante. Por ahora solo interesan los **números**.

Nota

Estos **comandos** hay que **aprendérselos**: los tenéis en el manual y en las guías, pero no se puede ir a buscarlos cada vez.

7.3 pinMode: ¿entrada o salida?

Arduino se puede dibujar como una **caja**. Esta caja puede **decidir** algo y mandar acciones hacia el exterior, o bien **escuchar** y dejarse influir por alguien desde el exterior.

Ejemplo

Boca y orejas. Para hablar usáis la **boca**: el sonido sale. Para escuchar usáis las **orejas**: la información entra. Son dos “periféricos” distintos.

- Un pin configurado como **oreja** es una **entrada**: en inglés **`INPUT`**.
- Un pin configurado como **boca**, con el que la placa le dice al mundo exterior qué hacer, es una **salida**: **`OUTPUT`**.

Definición

Antes de usar un pin hay que decir si es una **entrada o una salida**: no puedo decir que se encienda o se apague el pin 13 si es una oreja. Se usa el comando **`pinMode`**: entre los paréntesis el **número del pin** y luego `INPUT` u `OUTPUT`.

```
pinMode(13, OUTPUT);   // el pin 13 es una salida
```

En el código de Tinkercad este comando ya está, en la **línea 5**.

Atención

Cuando encendéis la placa, lo primero que pasa es que **todos los pines** son **entradas**: todo orejas. Si os **olvidáis del `pinMode`**, la lámpara **no funciona** como debe.

Nota

Dos comandos para recordar de memoria.

- **`pinMode`**: indica la **dirección** de la información, es decir si es Arduino quien decide qué hacer (salida) o si Arduino tiene que escuchar lo que dicen los demás (entrada).
- **`digitalWrite`**: para un pin de **salida**, decide si está **alto o bajo**.

7.4 Los dos bloques: setup y loop

Definición

Todo programa de Arduino tiene que contener **dos bloques de código**:

```
void setup()
{
  // aqui el codigo del setup
}

void loop()
{
  // aqui el codigo del loop
}
```

- Cada bloque es un **contenedor de código**: sus límites son las llaves, y dentro se escribe todo lo que haga falta.
- En nuestro programa el **`setup`** contiene solo el `pinMode(13, OUTPUT);`, el **`loop`** contiene **4 líneas** de código.

- Los dos bloques **tienen que estar siempre**: no se puede escribir un programa de Arduino al que le falte uno.
- Tiene que haber **uno solo** de cada tipo: no se pueden escribir dos `setup` o dos `loop`.

7.5 delay: esperar

Definición

Entre el encendido y el apagado está el comando **`delay`**: le dice a la máquina que se **detenga** durante el número de **milisegundos** escrito entre los paréntesis.

```
delay(1000);   // espera 1000 milisegundos
```

¿Cuánto son **1000 milésimas de segundo**? Un **segundo**: $1000 \times \frac{1}{1000}\,\text{s} = 1\,\text{s}$.

Ejemplo

Qué hace el programa, paso a paso. Enciende la lámpara, **espera un segundo**, la apaga, **espera un segundo**, y vuelve a empezar.

Atención

Setup una vez, loop para siempre.

- El **`setup`** se ejecuta **una sola vez**, al principio, cuando se enciende la placa.
- Terminado el setup, el **`loop`** se **repite continuamente**: mientras no se quite la alimentación, **no se detiene nunca**.

8 Qué recordar de esta lección

- El **microcontrolador** es el cerebro de la placa; en Arduino Uno es el **ATmega328P**.
- Arduino trabaja a **5 V**: un pin encendido (1) está a 5 V, apagado (0) a 0 V.
- Los pines digitales van del **0 al 13**, por lo tanto son **14**; los pines **0 y 1** (RX, TX) sirven para comunicarse con el ordenador: por ahora **no se usan**.
- **GND** es la referencia a **0 V**; tiene que llegar a **todos los periféricos**. En los esquemas tiene símbolos distintos.
- El **USB** sirve para programar (y alimenta durante las pruebas); después, la placa puede funcionar sola con una **batería**.
- En **Tinkercad**: Codice → de Blocchi a **Testo**; `LED_BUILTIN` → **13**.
- **`pinMode(pin, OUTPUT/INPUT)`**: salida (boca) o entrada (oreja).
- **`digitalWrite(pin, HIGH/LOW)`**: alto = encendido = 1, bajo = apagado = 0.
- **`delay(1000)`**: espera **1 segundo**; **`setup`** una sola vez, **`loop`** para siempre (uno de cada tipo, los dos obligatorios).

9 Glosario: palabras en italiano que se usan en clase

Italiano	Español
scheda (elettronica)	placa (electrónica)
microcontrollore	microcontrolador
piedino / pin / terminale	patita / pin / terminal
zampette	patitas
connettore	conector
filo	cable
tensione	tensión (voltaje)
acceso / spento	encendido / apagado
alto / basso	alto / bajo
massa / riferimento	masa / referencia
terra	tierra
schema elettrico	esquema eléctrico
alimentazione	alimentación
batteria	batería
lampadina	lámpara, lamparita
lampeggiare	parpadear
pulsante	pulsador
ingresso / uscita	entrada / salida
bocca / orecchie	boca / orejas
graffe	llaves
tonde	paréntesis
virgola / punto e virgola	coma / punto y coma
riga	línea
millisecondi	milisegundos
Crea circuito	Crear circuito
Codice / Blocchi / Testo	Código / Bloques / Texto
Avvia simulazione	Iniciar simulación
dispensa	guía (apuntes del profesor)